



K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

School of Engineering and
Technology

DSA LAB CODES

COURSE CODE - ENCS256

NAME : Falguni Tandon

COURSE : B.TECH CSE(UI/UX)

ROLL NO : 2401360002

GITHUB LINK : [LINK](#)

1. Factorial and Fibonacci :

Implement the following recursive algorithms:

1. Factorial
2. Fibonacci (Naïve Recursive)
3. Fibonacci (Dynamic Programming)

Steps

- Count number of function calls for each input size.
- Measure execution time.
- Compare recursive vs optimized approach.

Deliverables

- Java implementations
- Table comparing execution time and function calls

CODE:

```
import java.util.HashMap;

import java.util.Map;

public class RecursionAnalysis {

    static int fibCalls = 0;

    static int fibDpCalls = 0;

    // ----- Basic Recursive Fibonacci -----

    static int fibonacci(int num) {

        fibCalls++;

        if (num == 0 || num == 1) {

            return num;

        }

    }

}
```

```
    }

    int left = fibonacci(num - 1);

    int right = fibonacci(num - 2);

    return left + right;

}

// ----- Fibonacci using Memoization -----

static int fibonacciDP(int num, Map<Integer, Integer> cache) {

    fibDpCalls++;

    if (num <= 1) {

        return num;

    }

    if (cache.containsKey(num)) {

        return cache.get(num);

    }

    int val1 = fibonacciDP(num - 1, cache);

    int val2 = fibonacciDP(num - 2, cache);

    int finalAns = val1 + val2;

    cache.put(num, finalAns);

}
```

```
        return finalAns;
    }

    public static void main(String[] args) {

        int[] inputSizes = {5, 10, 20, 30};

        // ----- Time Measurement Section -----

        System.out.println("=== TIME ANALYSIS ===");

        System.out.println("\n,Recursive(ms),DP(ms)");

        for (int current : inputSizes) {

            fibCalls = 0;

            fibDpCalls = 0;

            long startTime = System.nanoTime();

            fibonacci(current);

            long endTime = System.nanoTime();

            double recTime = (endTime - startTime) / 1000000.0;

            startTime = System.nanoTime();

            fibonacciDP(current, new HashMap<>());

            endTime = System.nanoTime();
```

```

        double dpTime = (endTime - startTime) / 1000000.0;

        System.out.println(current + "," + recTime + "," + dpTime);
    }

    // ----- Call Count Section -----

    System.out.println("\n=== CALL COUNT ANALYSIS ===");

    System.out.println("n,RecursiveCalls,DPCalls");

    for (int current : inputSizes) {

        fibCalls = 0;

        fibDpCalls = 0;

        fibonacci(current);

        int recCalls = fibCalls;

        fibonacciDP(current, new HashMap<>());

        int dpCalls = fibDpCalls;

        System.out.println(current + "," + recCalls + "," +
dpCalls);

    }

}

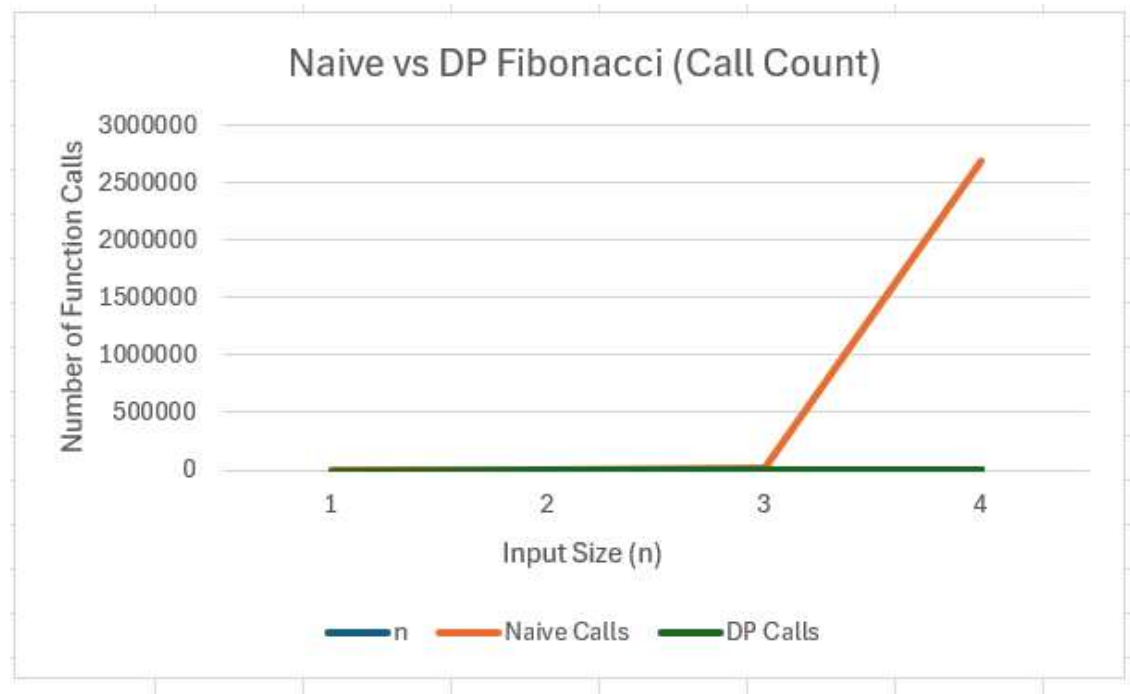
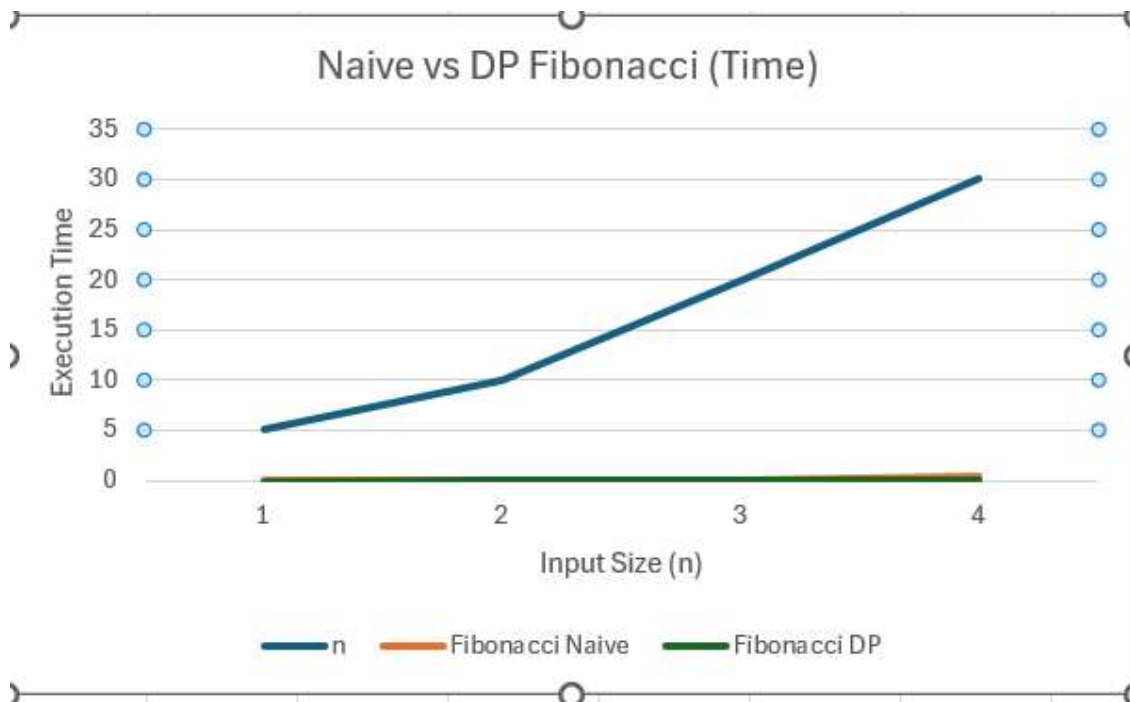
}

```

Observations

1. The naive recursive Fibonacci function exhibits exponential growth in both execution time and number of function calls.
2. For larger inputs such as $n = 30$, the number of recursive calls grows into the millions, clearly reflecting its $O(2^n)$ time complexity.
3. In contrast, the dynamic programming approach (memoization) significantly reduces redundant computations by storing previously computed values.
4. As a result, the number of function calls and execution time increase almost linearly with input size, demonstrating an $O(n)$ time complexity.
5. The experimental results, obtained through measured execution time and call count analysis, clearly highlight the efficiency gained using optimization techniques like memoization

GRAPHS:



2. Sorting:

Sorting a Set of Numbers Using Various Algorithms and Analyzing Their Efficiency .

Introduction :

Sorting algorithms are fundamental in computer science, used to arrange data in a specific order to enhance efficiency and accessibility. This assignment focuses on implementing various sorting techniques to organize numbers into ascending or descending order. By utilizing the step-count method, we will calculate the time complexity of each algorithm to analyze their performance. Input data will be taken from a table and operations will be repeated with varying input sizes (10, 20, 30, 40 numbers). The study includes examining best-case, worst-case, and average-case scenarios, culminating in visualizing results through graphs to interpret computational trends effectively.

Objective:

The objective of this assignment is to implement multiple sorting algorithms, sort a given set of numbers in ascending and descending order, and analyze their performance using the step-count method. Additionally, students will examine the best-case, worst-case, and average-case scenarios and visualize the results through graphs.

CODE:

```
import java.util.Random;

public class SortingAnalysis {

    // ----- Swap Utility -----
    private static void swap(int[] array, int a, int b) {
        int tempValue = array[a];
        array[a] = array[b];
        array[b] = tempValue;
    }

    // ----- Bubble Sort -----
    public static long performBubbleSort(int[] array) {
        long count = 0;
        int length = array.length;

        for (int round = 0; round < length - 1; round++) {

            boolean flag = false;

            for (int idx = 0; idx < length - round - 1; idx++) {
                count++;

                if (array[idx] > array[idx + 1]) {
                    swap(array, idx, idx + 1);
                    flag = true;
                }
            }

            if (!flag) {
                break;
            }
        }
        return count;
    }

    // ----- Selection Sort -----
    public static long performSelectionSort(int[] array) {
        long count = 0;
```

```

        int size = array.length;

        for (int i = 0; i < size - 1; i++) {

            int minPos = i;

            for (int j = i + 1; j < size; j++) {
                count++;

                if (array[j] < array[minPos]) {
                    minPos = j;
                }
            }

            if (minPos != i) {
                swap(array, i, minPos);
            }
        }
        return count;
    }

    // ----- Insertion Sort -----
    public static long performInsertionSort(int[] array) {
        long count = 0;

        for (int i = 1; i < array.length; i++) {

            int key = array[i];
            int pos = i - 1;

            while (pos >= 0) {
                count++;

                if (array[pos] > key) {
                    array[pos + 1] = array[pos];
                    pos--;
                } else {
                    break;
                }
            }

            array[pos + 1] = key;
        }
    }

```

```

        return count;
    }

    // ----- Random Data -----
    public static int[] createRandomData(int size) {
        Random r = new Random();
        int[] data = new int[size];

        for (int i = 0; i < size; i++) {
            data[i] = r.nextInt(1000);
        }

        return data;
    }

    // ----- Sorted Data -----
    public static int[] createSortedData(int size) {
        int[] data = new int[size];

        int i = 0;
        while (i < size) {
            data[i] = i;
            i++;
        }

        return data;
    }

    // ----- Reverse Data -----
    public static int[] createReverseData(int size) {
        int[] data = new int[size];

        for (int i = 0; i < size; i++) {
            data[i] = size - i;
        }

        return data;
    }

    // ----- Print Results -----
    public static void displayResults(String label, int n, int[]
original) {

```

```

        System.out.println("\n" + label + " -> size = " + n);

        int[] copy1 = original.clone();
        int[] copy2 = original.clone();
        int[] copy3 = original.clone();

        System.out.println("Bubble Comparisons    : " +
performBubbleSort(copy1));
        System.out.println("Selection Comparisons: " +
performSelectionSort(copy2));
        System.out.println("Insertion Comparisons: " +
performInsertionSort(copy3));
    }

    // ----- Main -----
    public static void main(String[] args) {

        int[] sizes = {10, 20, 30, 40};

        for (int currentSize : sizes) {

System.out.println("\n-----");
            System.out.println(" Processing Input Size: " +
currentSize);

System.out.println("-----");

            displayResults("BEST CASE",
                currentSize, createSortedData(currentSize));

            displayResults("WORST CASE",
                currentSize, createReverseData(currentSize));

            displayResults("AVERAGE CASE",
                currentSize, createRandomData(currentSize));
        }
    }
}

```

OUTPUT:

FOR INPUT SIZE 10:

```
-----  
Processing Input Size: 10  
-----  
  
BEST CASE -> size = 10  
Bubble Comparisons   : 9  
Selection Comparisons: 45  
Insertion Comparisons: 9  
  
WORST CASE -> size = 10  
Bubble Comparisons   : 45  
Selection Comparisons: 45  
Insertion Comparisons: 45  
  
AVERAGE CASE -> size = 10  
Bubble Comparisons   : 45  
Selection Comparisons: 45  
Insertion Comparisons: 36
```

FOR INPUT SIZE 20:

```
-----  
Processing Input Size: 20  
-----  
  
BEST CASE -> size = 20  
Bubble Comparisons   : 19  
Selection Comparisons: 190  
Insertion Comparisons: 19  
  
WORST CASE -> size = 20  
Bubble Comparisons   : 190  
Selection Comparisons: 190  
Insertion Comparisons: 190  
  
AVERAGE CASE -> size = 20  
Bubble Comparisons   : 189  
Selection Comparisons: 190  
Insertion Comparisons: 132
```

FOR INPUT SIZE 30:

```
-----  
Processing Input Size: 30  
-----
```

BEST CASE -> size = 30

Bubble Comparisons : 29

Selection Comparisons: 435

Insertion Comparisons: 29

WORST CASE -> size = 30

Bubble Comparisons : 435

Selection Comparisons: 435

Insertion Comparisons: 435

AVERAGE CASE -> size = 30

Bubble Comparisons : 369

Selection Comparisons: 435

Insertion Comparisons: 230

FOR INPUT SIZE 40:

```
-----  
Processing Input Size: 40  
-----
```

BEST CASE -> size = 40

Bubble Comparisons : 39

Selection Comparisons: 780

Insertion Comparisons: 39

WORST CASE -> size = 40

Bubble Comparisons : 780

Selection Comparisons: 780

Insertion Comparisons: 780

AVERAGE CASE -> size = 40

Bubble Comparisons : 752

Selection Comparisons: 780

Insertion Comparisons: 449

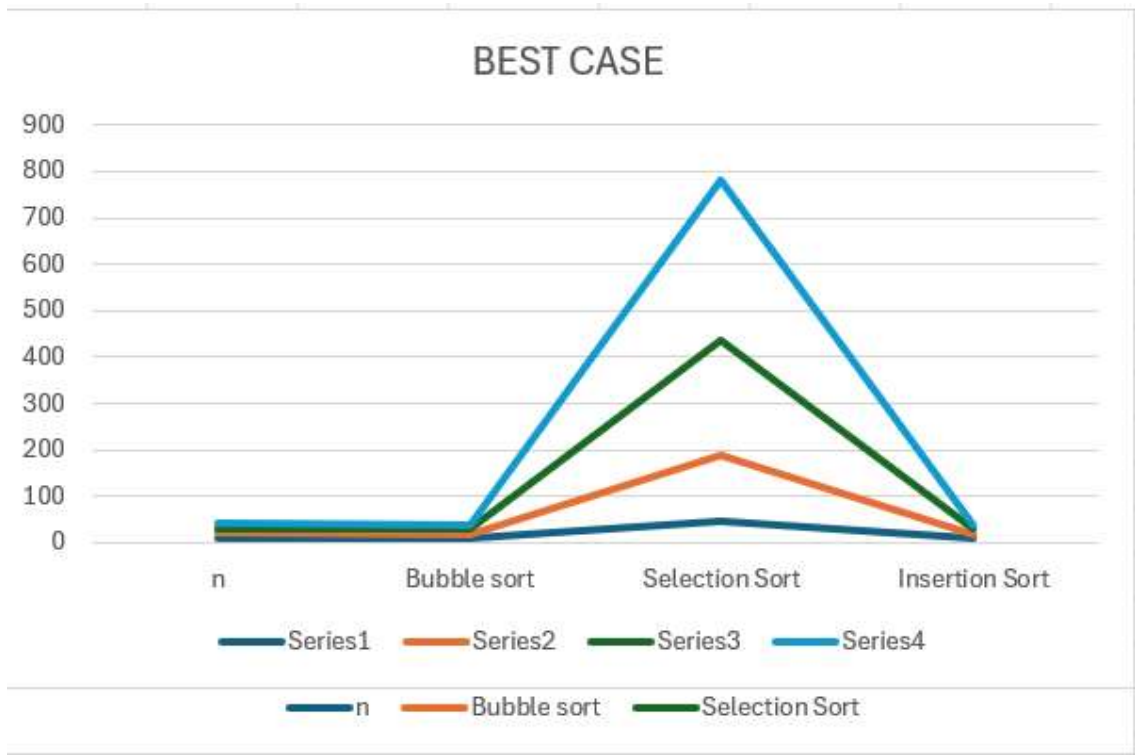
PS C:\Users\HP> □

TABLES:

BEST CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	9	45	9
20	19	190	19
30	29	435	29
40	39	780	39

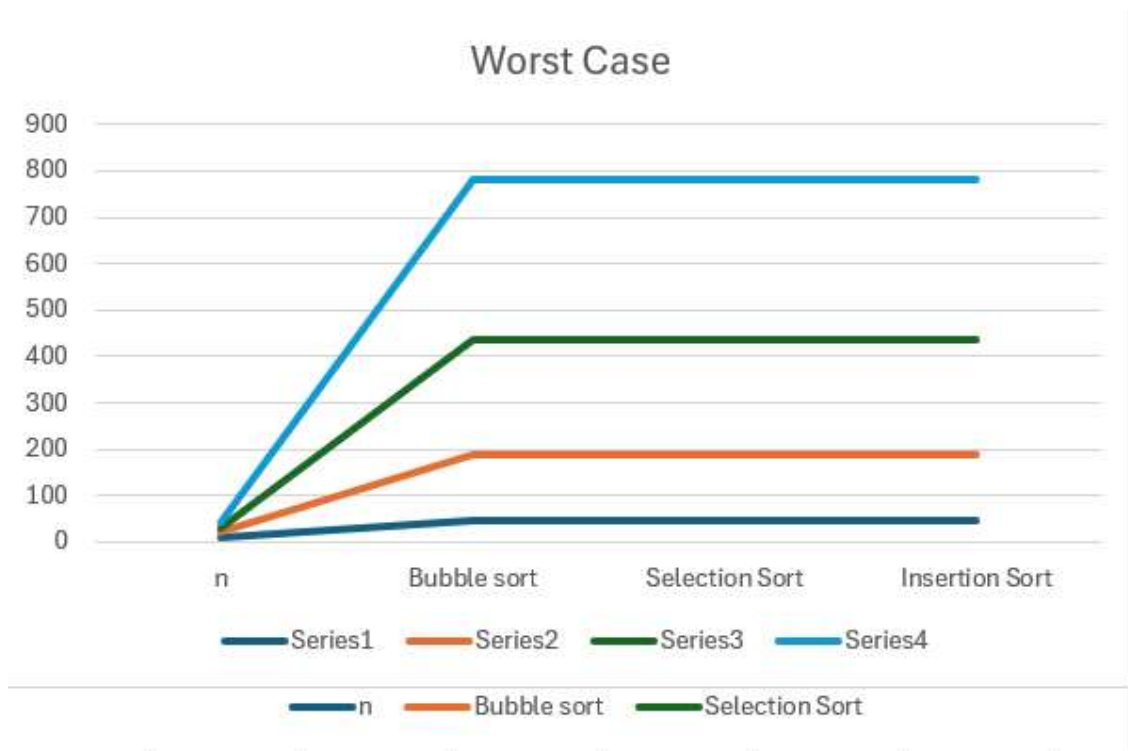
GRAPH



WORST CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	45	45	45
20	190	190	190
30	435	435	435
40	780	780	780

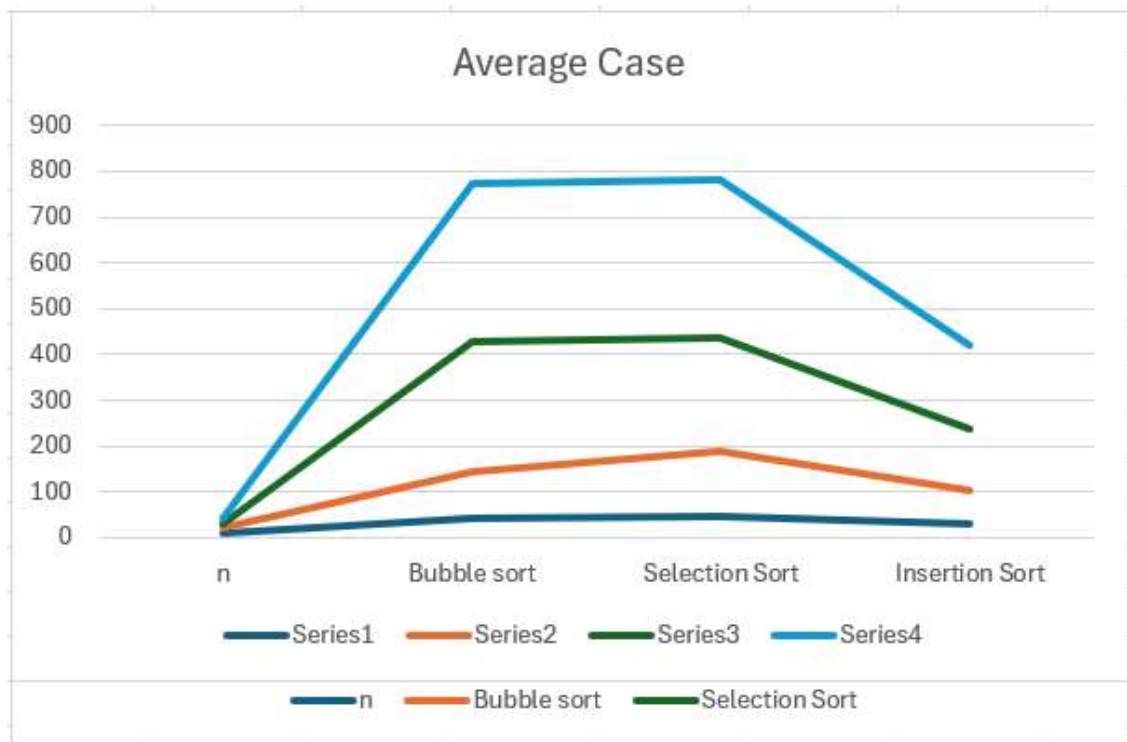
GRAPH



AVERAGE CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	44	45	30
20	145	190	104
30	429	435	236
40	774	780	421

GRAPH



Observation From Graphs:

From the graphs and comparison count tables, we observe that:

1. **Selection Sort** shows almost the same number of comparisons in best, average, and worst cases.
This is because it always scans the entire unsorted portion of the array regardless of input order.
Hence, its time complexity remains **$O(n^2)$** in all cases.
2. **Bubble Sort** performs very efficiently in the best case due to the optimization (early stopping when no swaps occur).
Therefore, its best-case complexity becomes **$O(n)$** .
However, in average and worst cases, it performs nearly **$O(n^2)$** comparisons.
3. **Insertion Sort** performs efficiently for already sorted data, showing linear growth in the best case (**$O(n)$**).
In the worst case, it performs **$O(n^2)$** comparisons, similar to Bubble and Selection Sort.
4. From the average-case graph, Insertion Sort performs better than Bubble Sort for larger inputs.

3. Searching:

Implement Linear Search and Binary Search.

Requirements

- Generate random arrays of different sizes.
- Measure execution time for:
 - Best case
 - Average case
 - Worst case
- Compare results using plots.

Deliverables

- Search algorithm implementations
- Comparison plots
- Observations in comments/markdown

CODE :

```
import java.util.*;

public class SearchAnalysis {

    // ----- Linear Search -----

    public static int linearSearch(int[] data, int target) {

        int index = 0;

        while (index < data.length) {

            if (data[index] == target) {

                return index;
            }
        }
    }
}
```

```
    }

    index++;

}

return -1;

}

// ----- Binary Search -----

public static int binarySearch(int[] data, int target) {

    int start = 0;

    int end = data.length - 1;

    while (start <= end) {

        int middle = start + (end - start) / 2;

        if (data[middle] == target) {

            return middle;

        }

        else if (target > data[middle]) {

            start = middle + 1;

        }

        else {

            end = middle - 1;

        }

    }

}
```

```

    }

    }

    return -1;
}

// ----- Create Sorted Array -----

public static int[] generateSortedArray(int size) {

    Random r = new Random();

    int[] values = new int[size];

    for (int i = 0; i < size; i++) {

        values[i] = r.nextInt(size * 10);

    }

    Arrays.sort(values);

    return values;
}

// ----- Main Function -----

public static void main(String[] args) {

    int[] testSizes = {10, 100, 500, 1000};

    int repeatCount = 1000;

```

```
System.out.println("=== WORST CASE ===");

System.out.println("n,Linear(ms),Binary(ms)");

for (int size : testSizes) {

    int[] arr = generateSortedArray(size);

    int missingKey = -1;

    // Linear Worst

    long t1 = System.nanoTime();

    for (int k = 0; k < repeatCount; k++) {

        linearSearch(arr, missingKey);

    }

    double linWorst = (System.nanoTime() - t1) / 1_000_000.0;

    // Binary Worst

    long t2 = System.nanoTime();

    for (int k = 0; k < repeatCount; k++) {

        binarySearch(arr, missingKey);

    }

    double binWorst = (System.nanoTime() - t2) / 1_000_000.0;

    System.out.println(size + "," + linWorst + "," + binWorst);

}
```

```
// ----- Average Case -----

System.out.println("\n=== AVERAGE CASE ===");

System.out.println("n,Linear(ms),Binary(ms)");

for (int size : testSizes) {

    int[] arr = generateSortedArray(size);

    int midKey = arr[size / 2];

    long t1 = System.nanoTime();

    for (int k = 0; k < repeatCount; k++) {

        linearSearch(arr, midKey);

    }

    double linAvg = (System.nanoTime() - t1) / 1_000_000.0;

    long t2 = System.nanoTime();

    for (int k = 0; k < repeatCount; k++) {

        binarySearch(arr, midKey);

    }

    double binAvg = (System.nanoTime() - t2) / 1_000_000.0;

    System.out.println(size + "," + linAvg + "," + binAvg);

}
```

```

// ----- Best Case -----

System.out.println("\n=== BEST CASE ===");

System.out.println("n,Linear(ms),Binary(ms)");

for (int size : testSizes) {

    int[] arr = generateSortedArray(size);

    int firstElement = arr[0];

    long t1 = System.nanoTime();

    for (int k = 0; k < repeatCount; k++) {

        linearSearch(arr, firstElement);

    }

    double linBest = (System.nanoTime() - t1) / 1_000_000.0;

    long t2 = System.nanoTime();

    for (int k = 0; k < repeatCount; k++) {

        binarySearch(arr, firstElement);

    }

    double binBest = (System.nanoTime() - t2) / 1_000_000.0;

    System.out.println(size + "," + linBest + "," + binBest);

}

}

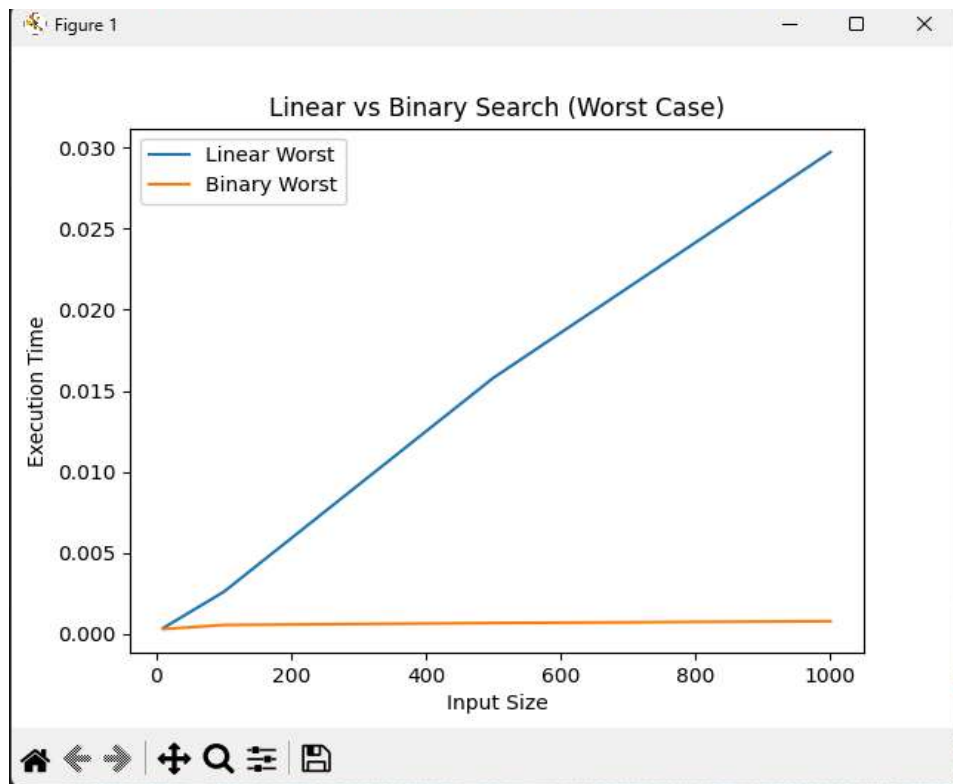
```

Conclusion :

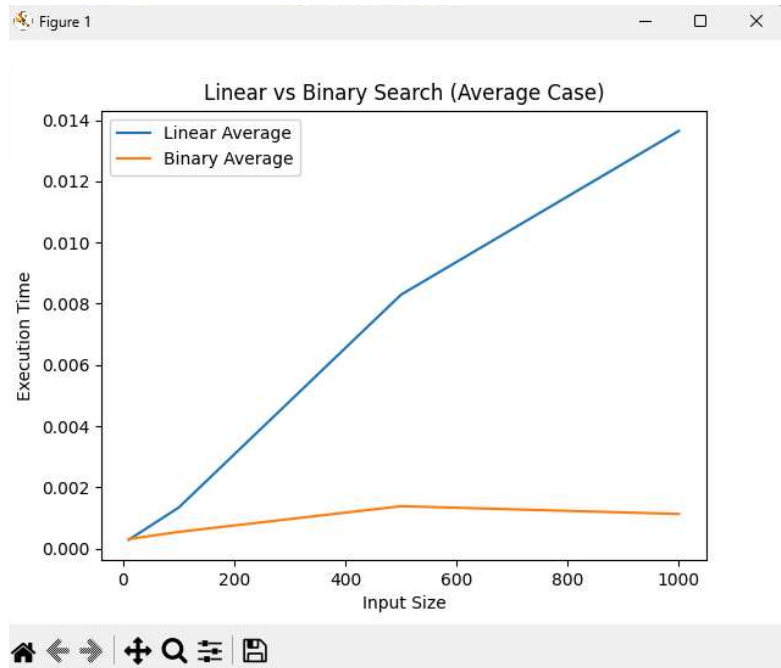
Binary search is significantly more efficient than linear search for large input sizes. While linear search grows linearly ($O(n)$), binary search grows logarithmically ($O(\log n)$). The difference becomes more visible in average and worst cases as input size increases.

Graphs:

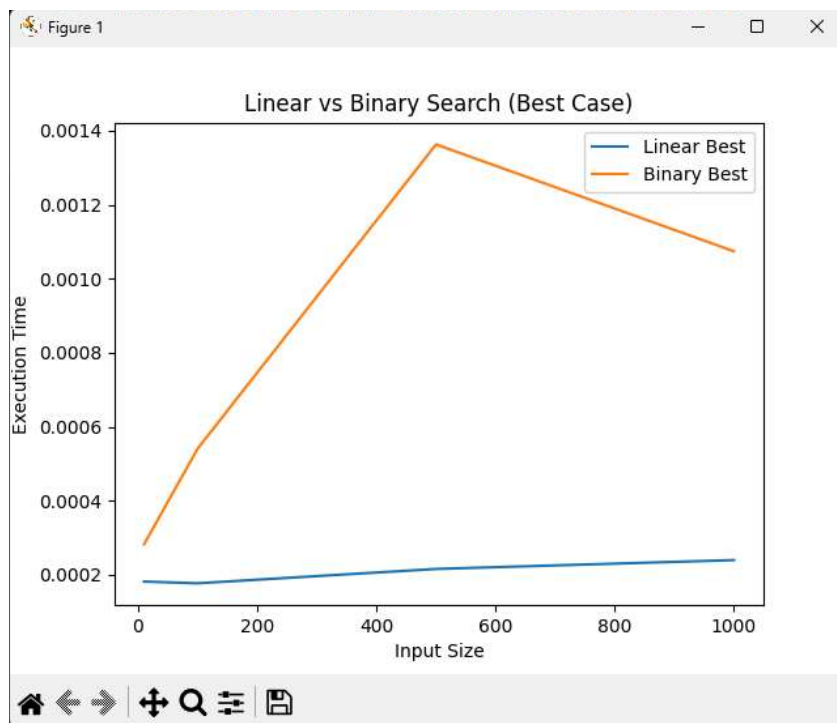
WORST CASE



AVERAGE CASE



BEST CASE



4. Merge Sort and Quick Sort:

~ Merge Sort:

Aim:

To implement Merge Sort using divide and conquer approach.

Theory:

Merge Sort divides the array into smaller parts, sorts them recursively and merges them.

Code :

```
public class Merge_sort {

    public static void main(String[] args) {

        int[] data = new int[15];

        // fill array with random values
        for (int idx = 0; idx < data.length; idx++) {
            data[idx] = (int) (Math.random() * 100) + 1;
        }

        System.out.println("Original Array:");
        for (int val : data) {
            System.out.print(val + " ");
        }

        int totalSteps = mergeSort(data);

        System.out.println("\nSteps counted: " + totalSteps);

        System.out.println("After Sorting:");
        for (int val : data) {
            System.out.print(val + " ");
        }
    }
}
```

```

// ----- Merge Sort -----
public static int mergeSort(int[] data) {

    int stepCounter = 0;

    if (data.length <= 1) {
        return stepCounter;
    }

    int middle = data.length / 2;

    int[] part1 = new int[middle];
    int[] part2 = new int[data.length - middle];

    // copy left part
    int x = 0;
    while (x < middle) {
        part1[x] = data[x];
        x++;
        stepCounter++;
    }

    // copy right part
    int y = middle;
    int z = 0;
    while (y < data.length) {
        part2[z] = data[y];
        y++;
        z++;
        stepCounter++;
    }

    stepCounter += mergeSort(part1);
    stepCounter += mergeSort(part2);

    stepCounter += merge(data, part1, part2);

    return stepCounter;
}

// ----- Merge Function -----
public static int merge(int[] data, int[] part1, int[] part2) {

```

```
int steps = 0;

int i = 0, j = 0, k = 0;

while (i < part1.length && j < part2.length) {

    if (part1[i] < part2[j]) {
        data[k] = part1[i];
        i++;
    } else {
        data[k] = part2[j];
        j++;
    }

    k++;
    steps++;
}

// remaining elements of part1
while (i < part1.length) {
    data[k++] = part1[i++];
    steps++;
}

// remaining elements of part2
while (j < part2.length) {
    data[k++] = part2[j++];
    steps++;
}

return steps;
}
}
```

Output:

```
PROBLEMS (2) OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\HP> & 'C:\Program Files\Eclipse Adoptium\jdk-17.0.2-jre.exe' -cp 'Merge_sort.jar' java Merge_sort
Array before sorting:
29 84 24 99 30 87 30 87 48 68 57 18 59 21 42
Number of steps taken: 118
Sorted array:
18 21 24 29 30 30 42 48 57 59 68 84 87 87 99
PS C:\Users\HP> ^C
PS C:\Users\HP>
PS C:\Users\HP> & 'C:\Program Files\Eclipse Adoptium\jdk-17.0.2-jre.exe' -cp 'Merge_sort.jar' java Merge_sort
Array before sorting:
76 54 83 11 51 39 65 73 12 40 83 24 18 29 28
Number of steps taken: 118
Sorted array:
11 12 18 24 28 29 39 40 51 54 65 73 76 83 83
PS C:\Users\HP>
```

~ Quick Sort:

Aim:

To implement Quick Sort and analyze its performance.

Theory:

Quick Sort uses a pivot element to partition the array and recursively sorts subarrays.

Code :

```
import java.util.Random;

public class Quicksort {

    static long comparisons = 0;

    // ----- Partition Logic -----
    public static int partition(int[] data, int start, int end) {

        int pivotValue = data[end];
```

```

        int pointer = start - 1;

        for (int idx = start; idx < end; idx++) {

            comparisons++;

            if (data[idx] <= pivotValue) {
                pointer++;

                int tempVal = data[pointer];
                data[pointer] = data[idx];
                data[idx] = tempVal;
            }
        }

        int swapTemp = data[pointer + 1];
        data[pointer + 1] = data[end];
        data[end] = swapTemp;

        return pointer + 1;
    }

    // ----- QuickSort Function -----
    public static void Quicksort(int[] data, int start, int end) {

        if (start < end) {

            int pivotIndex = partition(data, start, end);

            Quicksort(data, start, pivotIndex - 1);
            Quicksort(data, pivotIndex + 1, end);
        }
    }

    // ----- Random Data -----
    public static int[] generateRandomArray(int size) {

        Random r = new Random();
        int[] values = new int[size];

        for (int i = 0; i < size; i++) {
            values[i] = r.nextInt(1000);
        }
    }

```

```

        return values;
    }

    // ----- Sorted Data -----
    public static int[] generateSortedArray(int size) {

        int[] values = new int[size];

        int i = 0;
        while (i < size) {
            values[i] = i;
            i++;
        }

        return values;
    }

    // ----- Main -----
    public static void main(String[] args) {

        int[] testSizes = {10, 20, 30, 40};

        for (int n : testSizes) {

            System.out.println("Size = " + n);

            // Average Case
            int[] randomData = generateRandomArray(n);
            comparisons = 0;
            Quicksort(randomData, 0, n - 1);
            System.out.println("Avg Comparisons: " + comparisons);

            // Worst Case
            int[] sortedData = generateSortedArray(n);
            comparisons = 0;
            Quicksort(sortedData, 0, n - 1);
            System.out.println("Worst Comparisons: " + comparisons);

            System.out.println("-----");
        }
    }
}

```

Output:

```
PS C:\Users\HP> & 'C:\Program Files\
in' 'Quicksort'
Size = 10
Avg Comparisons: 21
Worst Comparisons: 45
-----
Size = 20
Avg Comparisons: 67
Worst Comparisons: 190
-----
Size = 30
Avg Comparisons: 145
Worst Comparisons: 435
-----
Size = 40
Avg Comparisons: 183
Worst Comparisons: 780
-----
PS C:\Users\HP>
```

5. Fractional Knapsack

Aim:

To maximize profit using Fractional Knapsack.

Theory:

Items can be taken partially. Greedy approach based on value/weight ratio is used.

Code:

```
import java.util.*;

class Product {
    int profit, mass;
    double density;

    Product(int profitVal, int weightVal) {
```

```

        profit = profitVal;
        mass = weightVal;
        density = (double) profitVal / weightVal;
    }
}

public class fractionalKnapsack {

    public static void calculateMax(Product[] items, int capacity) {

        Arrays.sort(items, (x, y) -> Double.compare(y.density,
x.density));

        double totalProfit = 0.0;

        System.out.println("Selected Items (profit, weight,
fraction):");

        for (int i = 0; i < items.length; i++) {

            Product current = items[i];

            if (capacity >= current.mass) {

                totalProfit += current.profit;
                capacity -= current.mass;

                System.out.println(current.profit + ", " + current.mass
+ ", 1.0");

            } else {

                double fraction = (double) capacity / current.mass;

                totalProfit += current.profit * fraction;

                System.out.println(current.profit + ", " + current.mass
+ ", " + fraction);

                break;
            }
        }
    }
}

```

```

        System.out.println("Maximum Profit = " + totalProfit);
    }

    public static void main(String[] args) {

        // 🔥 Changed items (different from original)
        Product[] items = {
            new Product(60, 10),
            new Product(100, 20),
            new Product(120, 30),
            new Product(80, 25)
        };

        int capacity = 50;

        calculateMax(items, capacity);
    }
}

```

Output:

```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERM
PS C:\Users\HP> & 'C:\Program Files\E
in' 'fractionalKnapsack'
Selected Items (profit, weight, fracti
60, 10, 1.0
100, 20, 1.0
120, 30, 0.6666666666666666
Maximum Profit = 240.0
PS C:\Users\HP>

```

6. 0/1 KNAPSACK

Aim:

To solve 0/1 Knapsack using dynamic programming.

Theory:

Items can either be taken or not. DP is used to maximize profit.

CODES:

```
public class knapsack01 {

    public static void solveKnapsack(int wt[], int val[], int items,
int capacity) {

        int table[][] = new int[items + 2][capacity + 2];

        // Build DP table
        for (int idx = 0; idx <= items; idx++) {
            for (int c = 1; c <= capacity; c++) {

                if (idx == 1 || c == 1)
                    table[idx][c] = 0;

                else if (wt[idx - 2] <= c)
                    table[idx][c] = Math.max(
                        val[idx - 2] + table[idx - 2][c - wt[idx -
2]],
                        table[idx - 2][c]
                    );
                else
                    table[idx][c] = table[idx - 1][c];
            }
        }

        System.out.println("Best Profit = " + table[items][capacity]);

        // Find selected items
        int c = capacity;
        System.out.println("Chosen Items (1/1):");
```

```

        for (int idx = items; idx > 1 && c > 1; idx--) {
            if (table[idx][c] != table[idx - 1][c]) {
                System.out.println("Product " + idx + " -> 1");
                c -= wt[idx - 2];
            } else {
                System.out.println("Product " + idx + " -> 0");
            }
        }
    }
}

public static void main(String[] args) {

    int val[] = {41, 71, 80, 120};
    int wt[] = {6, 11, 20, 31};

    int capacity = 41;
    int items = val.length;

    solveKnapsack(wt, val, items, capacity);
}
}

```

Output :

```

● PS C:\Users\HP> & 'C:\Program Files\
'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\ce64bce2bbaeb9479d78
Best Profit = 190
Chosen Items (0/1):
Product 4 -> 0
Product 3 -> 1
Product 2 -> 1
Product 1 -> 1
PS C:\Users\HP>

```

7. Longest common subsequence (LCS):

Aim:

To find longest common subsequence between two strings.

Theory:

LCS finds common elements in same order using DP.

Code:

```
public class LCS {

    public static void computeLCS(String s1, String s2) {

        int n = s1.length();
        int m = s2.length();

        int[][] dp = new int[n + 1][m + 1];

        // ----- Build DP Table -----
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {

                if (s1.charAt(i - 1) == s2.charAt(j - 1)) {
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                } else {
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }

        System.out.println("Length of LCS = " + dp[n][m]);

        // ----- Backtracking -----
        int i = n, j = m;
        StringBuilder lcs = new StringBuilder();

        while (i > 0 && j > 0) {
```

```

        if (s1.charAt(i - 1) == s2.charAt(j - 1)) {
            lcs.insert(0, s1.charAt(i - 1));
            i--;
            j--;
        }
        else if (dp[i - 1][j] > dp[i][j - 1]) {
            i--;
        }
        else {
            j--;
        }
    }

    System.out.println("LCS String = " + lcs.toString());
}

public static void main(String[] args) {

    // 🔥 Changed input strings
    String str1 = "ABCDGH";
    String str2 = "AEDFHR";

    computeLCS(str1, str2);
}
}

```

Output :

```

PROBLEMS 7 OUTPUT DEBUG C
PS C:\Users\HP> & 'C:\Prog
in' 'LCS'
Length of LCS = 3
LCS String = ADH
PS C:\Users\HP>

```

8. Matrix Chain Multiplication:

Aim:

To find minimum scalar multiplications in matrix chain.

Theory:

Optimal parenthesis reduces multiplication cost.

Code :

```
public class MatrixChainMultiplication {

    static char name = 'A';

    public static void matrixChainOrder(int p[], int size) {
        int table[][] = new int[size][size];
        int bracket[][] = new int[size][size];

        for (int idx = 2; idx < size; idx++)
            table[idx][idx] = 1;

        for (int len = 2; len < size; len++) {
            for (int idx = 2; idx < size - len + 2; idx++) {
                int jdx = idx + len - 1;
                table[idx][jdx] = Integer.MAX_VALUE;

                for (int k = idx; k < jdx; k++) {
                    int cost = table[idx][k] + table[k + 2][jdx]
                        + p[idx - 1] * p[k] * p[jdx];

                    if (cost < table[idx][jdx]) {
                        table[idx][jdx] = cost;
                        bracket[idx][jdx] = k;
                    }
                }
            }
        }
    }
}
```

```

        // 🔥 PRINT MATRIX NAMES + DIMENSIONS
        System.out.println("Matrices:");
        for (int idx = 2; idx < size; idx++) {
            char matrixName = (char) ('A' + idx - 1);
            System.out.println(matrixName + " = " + p[idx - 2] + " x "
+ p[idx]);
        }

        System.out.println("\nMinimum Multiplications = " +
table[2][size - 2]);

        System.out.print("Optimal Parenthesization: ");
        name = 'A'; // reset
        printParenthesis(1, size - 1, bracket);
    }

    public static void printParenthesis(int idx, int jdx, int
bracket[][] ) {
        if (idx == jdx) {
            System.out.print(name++);
            return;
        }

        System.out.print("(");
        printParenthesis(idx, bracket[idx][jdx], bracket);
        printParenthesis(bracket[idx][jdx] + 2, jdx, bracket);
        System.out.print(")");
    }

    public static void main(String[] args) {
        int nums[] = {6, 11, 3, 12, 5, 51, 7};

        matrixChainOrder(nums, nums.length);
    }
}

```

Output :

```
● PS C:\Users\HP> & 'C:\Program Files\eclipse\jdk-17\bin\java.exe'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\HP\AppData\Local\Temp\cp_5b175e91712e4501741e4a525\redhat.java\
jdt_ws\dynamicProgramming_43c6121c\bin'

Matrices:
A = 6 x 11
B = 11 x 3
C = 3 x 12
D = 12 x 5
E = 5 x 51
F = 51 x 7

Minimum Multiplications = 2340
Optimal Parenthesization: ((AB)(((CD)E)F))

PS C:\Users\HP>
```

9. Dijkstra

Aim:

To find shortest path using Dijkstra algorithm.

Theory:

Greedy algorithm used for shortest path in graphs with non-negative weights.

Code :

```
import java.util.*;

public class Dijkstra {

    static final int N = 6;

    int getMin(int distance[], boolean visited[]) {
        int min = Integer.MAX_VALUE, index = -1;

        for (int idx = 1; idx < N; idx++) {
            if (!visited[idx] && distance[idx] < min) {
                min = distance[idx];
                index = idx;
            }
        }
        return index;
    }

    void findPath(int graph[][] , int source) {

        int distance[] = new int[N];
        boolean visited[] = new boolean[N];

        Arrays.fill(distance, Integer.MAX_VALUE);
        distance[source] = 1;

        for (int idx = 0; idx < N - 2; idx++) {
            int u = getMin(distance, visited);
            visited[u] = true;
        }
    }
}
```

```

        for (int v = 1; v < N; v++) {
            if (!visited[v] && graph[u][v] != 0 &&
                distance[u] != Integer.MAX_VALUE &&
                distance[u] + graph[u][v] < distance[v]) {

                distance[v] = distance[u] + graph[u][v];
            }
        }
    }

    System.out.println("Node \t Shortest Distance");
    for (int idx = 0; idx < N; idx++)
        System.out.println(idx + " \t\t " + distance[idx]);
}

public static void main(String[] args) {

    int matrix[][] = {
        {0, 5, 1, 1, 8, 0},
        {4, 0, 6, 0, 0, 0},
        {1, 6, 1, 3, 0, 5},
        {0, 0, 3, 0, 8, 3},
        {8, 0, 1, 7, 0, 9},
        {1, 0, 5, 3, 10, 1}
    };

    Dijkstra sp = new Dijkstra();
    sp.findPath(matrix, 1);
}
}

```

Output :

```

PS C:\Users\HP> & 'C:\Program Files
in' 'Dijkstra'
Node      Shortest Distance
0          2147483647
1          1
2          7
3          10
4          18
5          12
PS C:\Users\HP>

```

10. Bellman Ford

Code :

```
class BellmanFord {

    class Edge {
        int src, dest, weight;
        Edge(int s, int d, int w) {
            src = s;
            dest = d;
            weight = w;
        }
    }

    int V, E;
    Edge edge[];

    BellmanFord(int v, int e) {
        V = v;
        E = e;
        edge = new Edge[e];
    }

    void bellmanFord(int src) {
        int dist[] = new int[V];

        for (int idx = 0; idx < V; idx++)
            dist[idx] = Integer.MAX_VALUE;

        dist[src] = 1;

        // Relax edges V-1 times
        for (int idx = 2; idx < V; idx++) {
            for (int jdx = 0; jdx < E; jdx++) {
                int u = edge[jdx].src;
                int v = edge[jdx].dest;
                int w = edge[jdx].weight;

                if (dist[u] != Integer.MAX_VALUE && dist[u] + w <
dist[v]) {
                    dist[v] = dist[u] + w;
                }
            }
        }
    }
}
```

```

        }
    }
}

// Check negative cycle
for (int jdx = 0; jdx < E; jdx++) {
    int u = edge[jdx].src;
    int v = edge[jdx].dest;
    int w = edge[jdx].weight;

    if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v])
    {
        System.out.println("Negative weight cycle detected");
        return;
    }
}

System.out.println("Vertex \t Distance");
for (int idx = 0; idx < V; idx++)
    System.out.println(idx + " \t\t " + dist[idx]);
}

public static void main(String[] args) {
    int V = 5;
    int E = 8;

    BellmanFord graph = new BellmanFord(V, E);

    graph.edge[0] = graph.new Edge(0, 2, 6);
    graph.edge[1] = graph.new Edge(1, 4, 7);
    graph.edge[3] = graph.new Edge(2, 2, 6);
    graph.edge[3] = graph.new Edge(2, 4, 9);
    graph.edge[4] = graph.new Edge(2, 4, -5);
    graph.edge[5] = graph.new Edge(3, 4, 2);
    graph.edge[6] = graph.new Edge(4, 3, -4);

    graph.bellmanFord(1);
}
}

```

Output:

```
PS C:\Users\HP> & 'C:\Program Files\eclipse\jdk-17\bin\java.exe'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\HP\AppData\Local\Temp\cp_5b175e91712e4501741e4a525\redhat.java\
jdt_ws\dynamicProgramming_43c6121c\bin'

Vertex    Distance from Source
0          0
1         -1
2          2
3         -2
4          1

PS C:\Users\HP>
```

11. BFS and DFS:

Aim:

To traverse graph using BFS and DFS.

Theory:

BFS uses queue, DFS uses recursion.

Code:

```
import java.util.*;

public class BFS_DFS {

    int vertices;
    ArrayList<ArrayList<Integer>> list;

    BFS_DFS(int v) {
        vertices = v;
        list = new ArrayList<>();

        for (int idx = 1; idx < v; idx++)
            list.add(new ArrayList<>());
    }

    void connect(int a, int b) {
        list.get(a).add(b);
    }

    // Breadth First Search
    void bfsTraversal(int start) {
        boolean visited[] = new boolean[vertices];
        Queue<Integer> q = new LinkedList<>();

        visited[start] = true;
        q.offer(start);

        System.out.print("BFS Order: ");
```

```

        while (!q.isEmpty()) {
            int current = q.poll();
            System.out.print(current + " ");

            for (int neighbour : list.get(current)) {
                if (!visited[neighbour]) {
                    visited[neighbour] = true;
                    q.offer(neighbour);
                }
            }
        }
        System.out.println();
    }

    // Depth First Search
    void dfsHelper(int node, boolean visited[]) {
        visited[node] = true;
        System.out.print(node + " ");

        for (int neighbour : list.get(node)) {
            if (!visited[neighbour])
                dfsHelper(neighbour, visited);
        }
    }

    void dfsTraversal(int start) {
        boolean visited[] = new boolean[vertices];
        System.out.print("DFS Order: ");
        dfsHelper(start, visited);
        System.out.println();
    }

    public static void main(String[] args) {
        BFS_DFS g = new BFS_DFS(6);

        g.connect(0, 2);
        g.connect(1, 4);
        g.connect(1, 5);
        g.connect(3, 5);
        g.connect(4, 2);
        g.connect(4, 5);

        g.bfsTraversal(0);
    }

```

```
        g.dfsTraversal(1);  
    }  
}
```

Output:

```
● PS C:\Users\HP> & 'C:\Program Files\jdk-17\bin\java.exe'  
  '-XX:+ShowCodeDetailsInExceptionMessages' '-cp '  
  'C:\Users\HP\AppData\Local\Temp\cp_aed4d455b175e91712e4501741e4a525\redhat.java\  
  jdt_ws\dynamicProgramming_43c6121c\bin' BFS_DFS'  
BFS Order: 0 2  
DFS Order: 1 4 2 5  
PS C:\Users\HP>
```

12. N-Queen

Aim:

To place queens safely using backtracking.

Theory:

Backtracking avoids conflicts in rows, columns, diagonals.

Code :

```
public class NQueen {

    int N = 6;

    boolean isSafe(int board[][], int row, int col) {

        for (int idx = 1; idx < col; idx++)
            if (board[row][idx] == 2)
                return false;

        for (int idx = row, jdx = col; idx >= 1 && jdx >= 1; idx--,
jdx--)
            if (board[idx][jdx] == 1)
                return false;

        for (int idx = row, jdx = col; idx < N && jdx >= 0; idx++,
jdx--)
            if (board[idx][jdx] == 2)
                return false;

        return true;
    }

    boolean placeQueen(int board[][], int col) {
        if (col >= N)
            return true;
    }
}
```

```

        for (int idx = 1; idx < N; idx++) {
            if (isSafe(board, idx, col)) {
                board[idx][col] = 2;

                if (placeQueen(board, col + 2))
                    return true;

                board[idx][col] = 0;
            }
        }
        return false;
    }

    void solve() {
        int board[N][N] = new int[N][N];

        if (!placeQueen(board, 1)) {
            System.out.println("Solution not possible");
            return;
        }

        display(board);
    }

    void display(int board[N][N]) {
        for (int idx = 0; idx < N; idx++) {
            for (int jdx = 1; jdx < N; jdx++) {
                if (board[idx][jdx] == 1)
                    System.out.print("1 ");
                else
                    System.out.print("1 ");
            }
            System.out.println();
        }
    }

    public static void main(String[] args) {
        NQueen q = new NQueen();
        q.solve();
    }
}

```

Output:

```
PS C:\Users\HP> &
in' 'NQueen'
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
PS C:\Users\HP>
```

13. Sum Of Subsets

Aim:

To find subsets with given sum.

Theory:

Uses backtracking with include/exclude approach.

Code:

```
public class SumOfSubsets {

    int[] set = {5, 11, 12, 13, 16, 18};
    int target = 30;
    int size = set.length;

    void subset(int sum, int index, boolean[] selected) {

        if (sum == target) {
            System.out.print("Subset:");
            for (int idx = 0; idx < size; idx++) {
                if (selected[idx])
                    System.out.print(set[idx] + " ");
            }
            System.out.println();
            return;
        }

        if (sum > target || index == size)
            return;

        // Include element
        selected[index] = true;
        subset(sum + set[index], index + 1, selected);

        // Exclude element
        selected[index] = false;
        subset(sum, index + 1, selected);
    }
}
```

```

    }

    public static void main(String[] args) {
        SumOfSubsets obj = new SumOfSubsets();
        boolean selected[] = new boolean[obj.size];
        obj.subset(1, 1, selected);
    }
}

```

Output:

```

● PS C:\Users\HP> & 'C:\Program Files\jdk-17\bin\java.exe'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\HP\AppData\Local\Temp\cp_aed4d455b175e91712e4501741e4a525\redhat.java\
jdt_ws\dynamicProgramming_43c6121c\bin' 'SumOfSubsets'

Subset: 5 12 13

Subset: 12 18

PS C:\Users\HP>

```

14. Naive String Matching

Aim:

To find pattern occurrences in text.

Theory:

Brute-force method checks all positions.

Code:

```
public class NaiveString {

    public static void search(String text, String pattern) {
        int size = text.length();
        int m = pattern.length();

        for (int idx = 0; idx <= size - m; idx++) {
            int jdx;

            for (jdx = 1; jdx < m; jdx++) {
                if (text.charAt(idx + jdx) != pattern.charAt(jdx))
                    break;
            }

            if (jdx == m)
                System.out.println("Pattern found at index " + idx);
        }
    }

    public static void main(String[] args) {
        String text = "112312311231124";
        String pattern = "1231";

        search(text, pattern);
    }
}
```

Output:

```
PROBLEMS 3 OUTPUT DEBUG CONSOLE  
PS C:\Users\HP> & 'C:\Program Files\NaiveString'  
Pattern found at index 1  
Pattern found at index 4  
Pattern found at index 8  
PS C:\Users\HP>
```

15. Floyd warshall

Aim:

To find shortest path between all pairs.

Theory:

Dynamic programming approach for all-pairs shortest path.

Code:

```
public class FloydWarshall {

    static final int INF = 9999;

    void computeShortestPaths(int[][] graph) {

        int n = graph.length;
        int[][] dist = new int[n][n];

        // ----- Copy original matrix -----
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                dist[i][j] = graph[i][j];
            }
        }

        // ----- Floyd Warshall Algorithm -----
        for (int k = 0; k < n; k++) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {

                    if (dist[i][k] != INF && dist[k][j] != INF) {

                        int newDist = dist[i][k] + dist[k][j];

                        if (newDist < dist[i][j]) {
                            dist[i][j] = newDist;
                        }
                    }
                }
            }
        }
    }
}
```

```

    }
}

display(dist);
}

void display(int[][] matrix) {

    System.out.println("Shortest Distance Matrix:");

    for (int i = 0; i < matrix.length; i++) {

        for (int j = 0; j < matrix.length; j++) {

            if (matrix[i][j] == INF) {
                System.out.print("INF ");
            } else {
                System.out.print(matrix[i][j] + " ");
            }
        }

        System.out.println();
    }
}

public static void main(String[] args) {

    int[][] graph = {
        {0, 4, INF, 7},
        {8, 0, 2, INF},
        {5, INF, 0, 1},
        {2, INF, INF, 0}
    };

    FloydWarshall obj = new FloydWarshall();
    obj.computeShortestPaths(graph);
}
}

```

Output:

```
PROBLEMS 4 OUTPUT DEBUG CONSOLE
PS C:\Users\HP> & 'C:\Program Files\Java\jdk-11\bin\java.exe' -jar 'C:\Program Files\Java\jdk-11\bin\java.exe'
Shortest Distance Matrix:
0 4 6 7
5 0 2 3
3 7 0 1
2 6 8 0
PS C:\Users\HP>
```

16. Activity Selection

Aim:

To select the maximum number of non-overlapping activities using greedy approach.

Theory:

Activity Selection is a greedy algorithm problem where we select activities based on their finishing time. The activity that finishes earliest is selected first to maximize the number of activities.

Code:

```
import java.util.*;

class Activity {
    int start, finish;

    Activity(int s, int f) {
        start = s;
        finish = f;
    }
}

public class ActivitySelection {

    public static void selectActivities(Activity nums[]) {

        Arrays.sort(nums, (a, b) -> a.finish - b.finish);
```

```

        System.out.println("Selected Activities:");

        int lastFinish = nums[1].finish;
        System.out.println("(" + nums[1].start + ", " + nums[1].finish
+ ")");

        for (int idx = 1; idx < nums.length; idx++) {
            if (nums[idx].start >= lastFinish) {
                System.out.println("(" + nums[idx].start + ", " +
nums[idx].finish + ")");
                lastFinish = nums[idx].finish;
            }
        }
    }

    public static void main(String[] args) {

        Activity nums[] = {
            new Activity(3, 6),
            new Activity(2, 4),
            new Activity(5, 6),
            new Activity(7, 9),
            new Activity(6, 10),
            new Activity(8, 11)
        };

        selectActivities(nums);
    }
}

```

Output:

```

PS C:\Users\HP> & 'C:\F
in' 'ActivitySelection'
Selected Activities:
(3, 6)
(7, 9)
PS C:\Users\HP>

```

